

# Lean 入门

从形式化证明到基础  
tactic

6 日课程 · D1 讲义

```
import Mathlib
```

```
#check Nat
```

```
#eval 2 + 3
```

```
example (P : Prop) : P -> P := by
```

```
  intro h
```

```
  exact h
```

目标：理解 Lean 为什么能检查证明，并能写出第一批可运行证明。

# 六日课程总览：每天学什么？

D1 · 7/13

## 从零到逻辑 + AI 工具

类型论简史、函数与类型、Prop、基础 tactic、rw/simp

D2 · 7/14

## Inductive type 与 class

inductive、structure、常用归纳类型、class/instance

D3 · 7/15

## 代数结构 + 组合 / 图

代数层级、范畴演示、Finset 计数、SimpleGraph

D4 · 7/16

## 分析结构 + 拓扑

实数与序、极限、微分、积分、拓扑与度量空间

D5 · 7/17

## 函数式编程

高阶函数、递归、循环表达、Monad、notation、程序

D6 · 7/20

## 算法形式化与验证

规范、不变量、终止性、复杂度、CS 库、验证软件

主线：先学 Lean 的语言与证明，再进入 Mathlib 的数学库，最后用项目把 AI 辅助与形式化验证连起来。

# Tutorial 与项目：每天怎么推进？

## 下午 tutorial

D1

环境安装；AI 工具配置；Mathlib 检索；ring/linarith/decide

D2

Inductive、structure、class 练习答疑

D3

代数 / 组合练习；项目选题咨询；分组

D4-D5

项目答疑；用 AI 起草，再用检索与 Lean 修复

D6

算法验证共性问题简评；项目路线收束

## 项目节奏

D3

布置小组项目选题并分组

7/18-19

周末小组自行推进，可全程使用 AI 工具

D6

围绕 spec、不变量、终止性、复杂度做共性反馈

课堂原则：AI 可以给候选，学生负责理解与拆解，Lean/Mathlib 负责可检查的证据。

# 课程地图

---

形式化证  
明与 AI

函数与类  
型

C-H 对应  
与 Prop

逻辑 tactic

rw / simp

总结与练  
习

# 开场故事：Knuth 的 Claude's Cycles



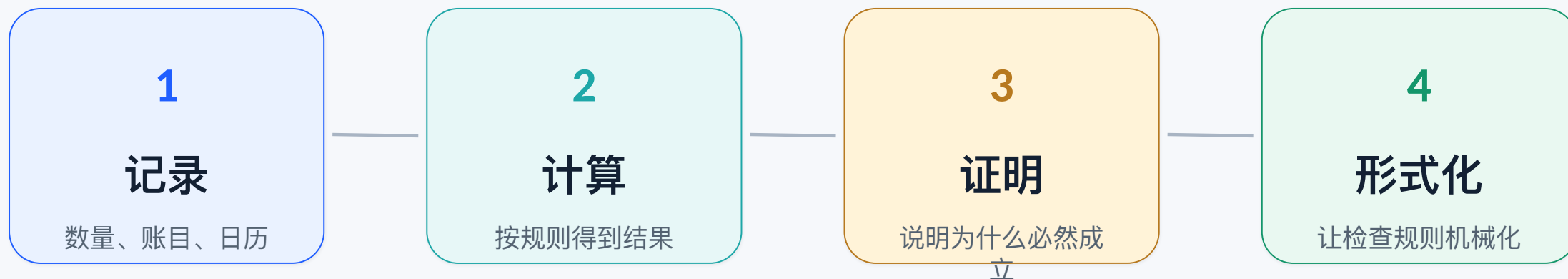
Donald Knuth

AI 搜索 + 人类证明 + Lean 形式化



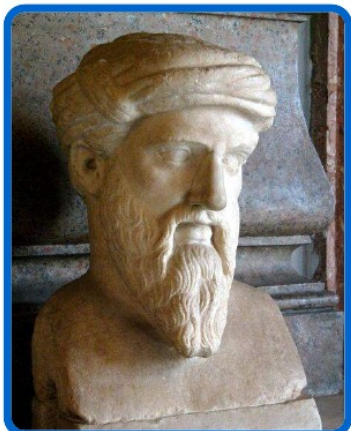
这不是“AI 自动完成数学”的故事，而是搜索、解释、计算、证明检查一起工作的现代图景。

# 历史主线：从记录到可检查证明



Lean 处在第四步：把“为什么成立”写成足够精确的对象，让小内核逐步检查。

## 早期数学：先有记录与计算



Pythagoras

- 古代数学首先服务于记忆：数量、土地、债务、天文与历法。
- 巴比伦、埃及等传统留下了结构化计算，而不一定以现代证明方式组织。
- 希腊传统把重点推进到“为什么必然成立”：例子之外要给出理由。

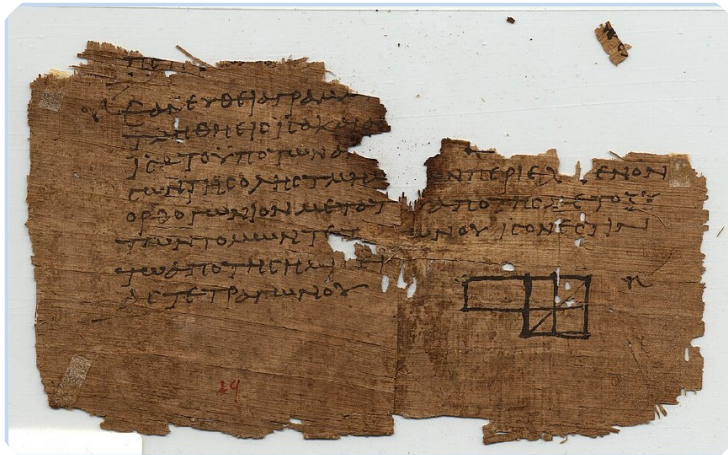
课堂抓手：形式化证明不是凭空出现的，它继承了“记录要准确、计算要可靠、理由要可复查”的长期需求。

# Euclid：把证明组织成依赖网络



Euclid

active c. 300 BCE



Elements 手稿片段

- 先给定义、公设、公理。
- 命题按依赖顺序排列。
- 图辅助理解；证明承担结论。

Euclid 的重要性不只是几何内容，而是“把数学知识排成可追踪的依赖链”。



# Al-Khwarizmi : 算法与知识保存



Al-Khwarizmi

c. 780-850

algebra

把问题转换为符号与规则

algorithm

可以反复执行的步骤

library

知识被保存、整理、复用



知识保存与翻译传统

从算法到库：Lean/Mathlib 继承的不是某一个定理，而是“把步骤和知识组织起来”的传统。

# 现代证明变长： Fermat 到 Wiles



**Fermat**

1607-1665



**Andrew Wiles**

1993 宣布, 1995 发表

$$a^n + b^n = c^n, n > 2$$

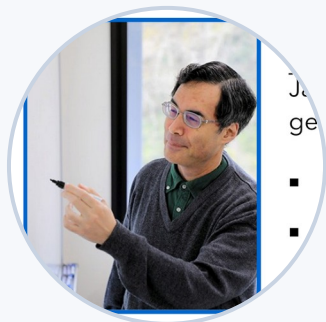
没有正整数解

**短陈述  
长背景**

半稳定情形  
模形式  
Galois 表示  
模性提升

Wiles 与 Taylor 的半稳定情形足以推出 FLT ; 完整模性定理由后续工作完成。

# 验证也会成为数学故事的一部分



**Mochizuki**

ABC / IUT



**Scholze**

2018 objection



**Stix**

2018 objection

- 2012：IUT 相关论文声称证明 ABC 猜想。
- 2018：Scholze-Stix 对关键步骤提出反对意见。
- 形式化项目试图把争议转为可检查的定义与依赖。

这类案例提醒我们：数学共同体需要的不只是答案，还需要能被共同检查的表达方式。

# Hilbert：能否把证明检查机械化？



David Hilbert

1862-1943

- 把数学放在明确的形式基础上。
- 用清楚的符号语言和推理规则表达证明。
- 希望证明这些基础本身的一致性。
- “机器检查证明”可以看成这个愿景的一个自然终点。

**Hilbert program 的核心冲动：不要只相信权威和直觉，要知道每一步到底按什么规则成立。**

# Principia Mathematica : 把推理写成符号



**Russell**

1872-1970



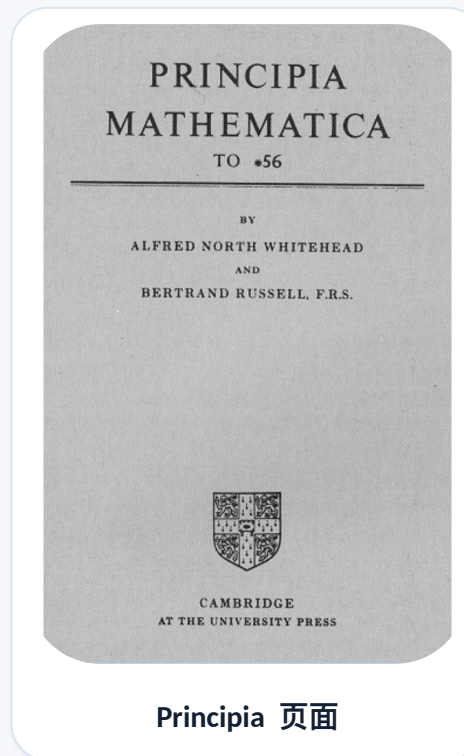
**Whitehead**

1861-1947

**1910-1913**

目标：用明确的符号  
规则推导数学。

著名的  $1 + 1 = 2$  结果  
要走过数百页。



Principia 页面

它让人看到形式化的力量，也让人看到完全显式化会带来巨大的书写成本。

# Gödel：限制不是终点，而是边界



**Kurt Gödel**

1906-1978

## 1931 不完全性定理

- 足够强且一致的形式系统，不能证明所有真命题。
- 这样的系统通常也不能在系统内部证明自身一致性。
- 但这不妨碍我们在指定系统中机械检查具体证明。

课堂需要的结论：Lean 不是“证明所有真理”的机器；它是“检查给定形式证明”的工具。

## Weyl：计算机作为数学的外部帮助



Hermann Weyl

1885-1955

1946

高性能计算机可能帮助越来越困难的数学工作。

搜索

机器寻找候选、枚举情况、发现模式。

检查

机器按规则验证对象是否真的成立。

之后的历史会分出两条支流：自动找证明，以及把证明写成可由机器检查的对象。

## 两条路线：找证明 vs 检查证明

### Proof Search

- 机器尝试找到推导
- 1956 Logic Theorist
- 今天： ATP / AI prover

### Proof Checking

- 证明对象已给出
- 1968 Automath
- 今天： Coq / Lean kernel

Lean 中二者会合： tactic 和 AI 可以帮助搜索，但最终必须通过内核检查。



## Automath：证明变成精确对象

```
ε * N := PN : set
ε * 0 := PN : N
ε * n := --- : N
n * s := PN : N
ε * 1 := s(0) : N
```

写成对象

证明不只是文字

逐行检查

每一步符合规则

小内核

Lean 的前身之一

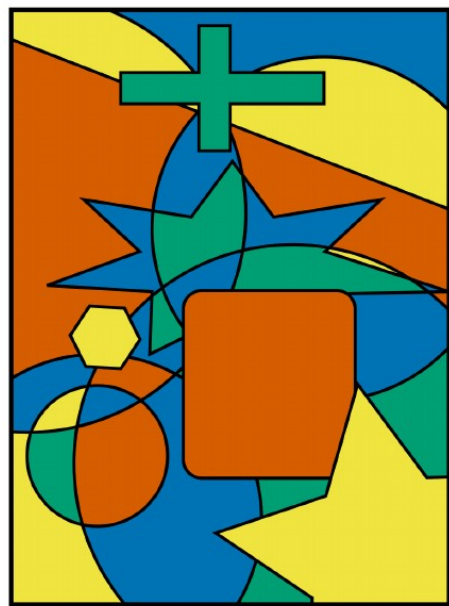
从“读懂一段文字”转为“检查一个对象”：这就是 Lean 内核工作的前身。

# 计算机辅助证明：四色定理



**Georges Gonthier**

2005 Coq 形式化



四颜色地图示意

- 1976：四色定理使用计算机完成大量分类检查。
- 问题：计算结果如何获得数学共同体信任？
- 2005：Gonthier 团队在 Coq 中完成机械检查。

这类例子把“计算正确吗？”和“证明可检查吗？”真正绑在一起。

# Flyspeck : 把复杂证明长期工程化



**Kepler**

1611 猜想



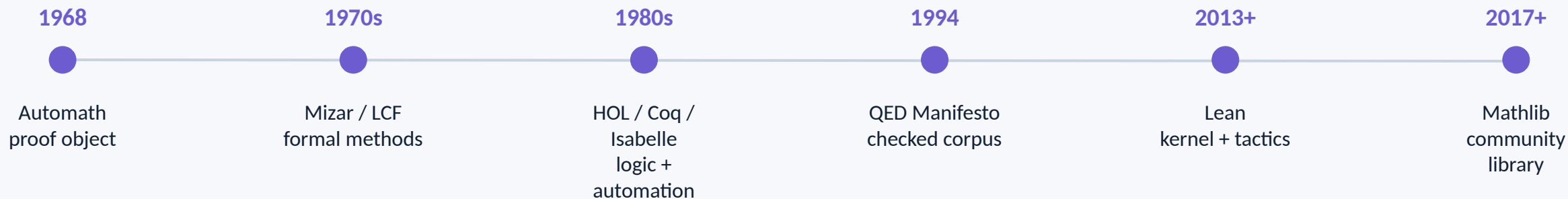
**Thomas Hales**

Flyspeck 项目

- 1998 : Hales 宣布 Kepler 球堆积猜想的计算机辅助证明。
- 2003-2014 : Flyspeck 项目用 HOL Light 和 Isabelle 完成形式化。
- 启发：形式化不是一次翻译，而是长期构建库、接口和证明脚本。

一个现代证明常常是一项工程：目标定理只是顶端，下面是大量可复用基础设施。

# 证明助手变成基础设施



Lean 不是凭空出现的新工具，而是形式基础、自动推理、证明检查和数学库传统汇合后的产物。

# 形式化证明的信任链条



Lean 检查“形式化后的命题是否有合法证明”；数学家仍要确认它表达了原来的数学意图。

# Lean : 语言、检查器、数学库



**Leonardo de Moura**

Lean creator



**Kevin Buzzard**

Xena / Mathlib advocate

- Lean 4 同时是函数式编程语言和定理证明器。
- 小内核检查证明项, tactic 帮我们构造证明项。
- Mathlib 把定义、定理、notation、typeclass 和 tactic 组织成库。

初学 Lean 的正确心态: 不是从零证明世界, 而是进入一个可复用、可检查的数学知识库。

# Mathlib：可复用的数学知识库



初学 Lean 时，真正要进入的是“库中心”的工作方式：先找已有定义和定理，再写自己的证明。

# Mathlib 的名字通常告诉你定理在说什么

Lean 里定理名不是自然语言，而是可搜索的地址。读名字时，先看 namespace，再看核心词。

**前缀**      Nat / Int / Set      对象或领域

**核心词**      add / mem / ext      运算或关系

**修饰词**      comm / left      方向或性质

```
#check Nat.add_comm  
#check Int.add_comm
```

```
#check Set.mem_union  
#check Finset.mem_union
```

**两个例子**

Nat.add\_comm : 加法交换律  
Set.mem\_union : 并集成员判定

课堂里遇到陌生名字时，先拆名字，再用 #check / exact? / apply? 验证它的类型。



## 不同领域会有同一个短名，完整名字才不会冲突

许多数学对象都有“外延性”定理，短名都可以叫 `ext`；Lean 用 `namespace` 把它们放进不同抽屉。

### Set.ext

两个集合相等  
只要所有元素的属于关系等价

$$\forall x, x \in s \leftrightarrow x \in t$$

### Finset.ext

两个有限集合相等  
也看每个元素是否同时属于

$$\forall x, x \in A \leftrightarrow x \in B$$

```
#check Set.ext  
#check Finset.ext
```

-- 同一个短名 `ext`，完整名不同

这就是为什么我们接下来要认识 `namespace`：它让大库里的短名可以重复，完整名仍然唯一。

# 文件组织命令控制名字从哪里来、在哪里可见

**import**

加载文件或库  
通常放在文件最上方

**namespace**

加完整名前缀  
Demo.double

**section**

限制局部作用域  
end 后局部变量消失

**open**

临时省略前缀  
open Nat 后可写  
zero\_add

```
-- import Mathlib

namespace Demo

def double (n : Nat) : Nat := n + n

end Demo

open Nat
#check zero_add
#check Nat.zero_add
```

经验法则：写库代码多用完整名；讲课和短证明中可以 open，但遇到歧义就退回完整名。

# AI for Math : Lean 在哪里?

探索

AI/ 程序寻找模式、构造或证明路线

Knuth Claude's Cycles

翻译

把自然语言目标变成形式陈述

IMO / research benchmarks

检查

Lean kernel 验证证明对象

AlphaProof / Lean formalization

本课学习的是很多 Lean prover 需要输出的目标语言，也是数学形式化项目的共同接口。

# 主流 AI 模型速览：商业前沿模型

截至 2026-06-30 的公开资料快照；模型更新很快，课堂中只看“选型直觉”。

## OpenAI

**GPT-5.6 preview ; GPT-5.5**

优势：复杂推理、coding、agent

生态：ChatGPT / API / Codex

适合：Lean 草稿与错误修复

## Anthropic

**Claude Fable 5 ; Opus 4.8 / Sonnet 5**

优势：长程 agentic coding

强项：复杂推理与文档任务

适合：长证明规划、代码库理解

## Google DeepMind

**Gemini 3.5 Flash ; 3.1 Pro preview**

优势：多模态、搜索、工具

生态：Google / Live / media

适合：检索与讲义素材

## xAI

**Grok 4.3 ; Grok Build 0.1**

优势：1M 上下文、工具调用

生态：Web / X Search、coding

适合：近期资料与快速对话

# 主流 AI 模型速览：开放权重与可部署生态

开放权重 / 可本地部署不等于“更弱”：它们常在成本、可控性、隐私和二次开发上更有优势。

## DeepSeek

**DeepSeek-V4-Pro / V4-Flash**

优势：1M 上下文、价格低  
双模式：thinking / non-thinking  
适合：批量草稿与 API 实验

## Qwen

**Qwen3.6-35B-A3B ; Qwen3.6 系列**

优势：开放权重、多语言 / 工具  
分线：Instruct-only / Thinking-only  
适合：中文、本地部署、代码代理

## Meta

**Llama 4 Scout / Maverick ; Behemoth preview**

优势：开放权重、原生多模态  
Scout：10M 上下文窗口  
适合：私有化、微调和实验

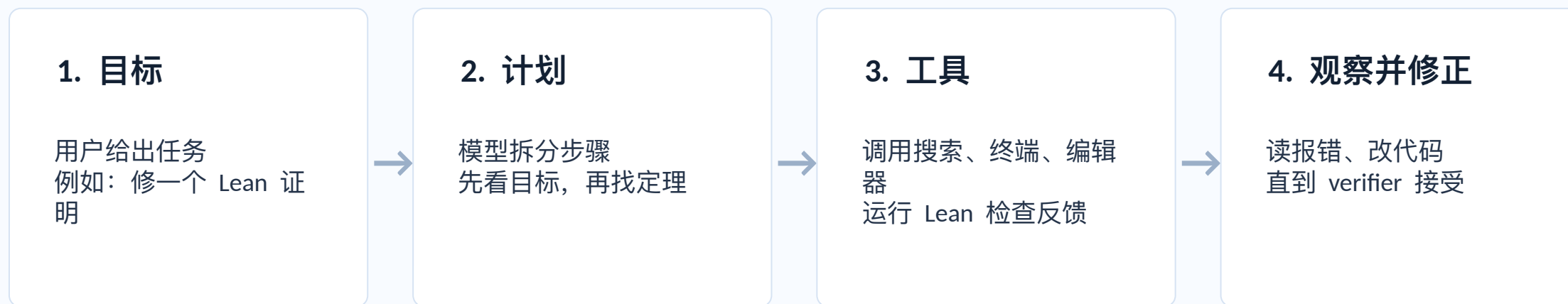
## Mistral

**Mistral Medium 3.5 ; Small 4**

优势：开放模型，coding/agent  
生态：OCR / 音频 / Devstral  
适合：轻量部署和专门任务

# Agent 把模型接成一个能做事的循环系统

模型负责理解、推理和生成； agent 负责把目标拆成步骤，调用工具，观察结果，再继续修正。



## 对 Lean 学习最重要的一点

agent 能更快尝试 lemma、改 tactic、运行检查，但它仍可能编造定理名或误读目标。Lean kernel 和 Mathlib 文档是最终裁判。

# 主流 agent 应用正在分成代码、浏览器和企业流程三类

截至 2026-07-04 的公开资料快照：有些公司推出完整应用，有些主要提供模型 API 或开源 agent 框架。

| 公司 / 生态     | 代表 agent 应用或能力                        | 课堂里的用法                  |
|-------------|---------------------------------------|-------------------------|
| OpenAI      | ChatGPT agent 、 Codex 、 Deep Research | 自然语言任务执行、代码修改、资料搜集与核对   |
| Anthropic   | Claude Code 、 Computer Use 、 tool use | 代码 agent 与电脑 / 浏览器操作型任务 |
| Google      | Jules 、 Gemini CLI 、 Agentspace       | 代码任务、命令行协作、企业知识流程       |
| GitHub      | Copilot coding agent                  | 从 issue 到 PR 的工程化代码修改流程 |
| xAI         | Grok 的实时搜索与 tool/function calling     | 需要实时信息时可做搜索型助手          |
| DeepSeek    | function calling API ， 常作低成本 agent 后端 | 适合本地 / 服务器端自建工具调用流程     |
| Qwen / Meta | Qwen-Agent ； Llama Stack              | 开源 / 可部署 agent 框架与本地实验  |
| Mistral     | Agents API                            | 企业场景下的工具调用、RAG 与自动化流程   |

判断 agent 好不好，不只看模型分数，还要看工具权限、日志透明度、能否复现、是否接入 verifier 。

## 在 Lean 课堂里，怎样选择 AI？

### 证明路线

选强推理 / 强编码模型。

让 AI 给 lemma 列表、证明分解、可能 tactic，而不是直接相信完整证明。

### 资料检索

选带搜索 / 引用的工具。

要求给出处、版本和日期；数学事实再回到 Mathlib / Lean 检查。

### 代码修复

选 coding/agent 能力强的模型。

把错误信息、目标状态、相关 import 一起喂给它，限制修改范围。

### 本地部署

选开放权重模型。

适合隐私、批量实验和课程环境，但仍要注意能力、成本与维护负担。

### 课堂原则

AI 负责提出候选；学生负责理解与拆解；Lean 内核负责给出可检查的最终证据。



# AI 在数学学习中擅长什么？

## 探索

快速生成例子、猜想、证明路线，帮助打开思路。

## 解释

把陌生定义换一种说法，给出类比和小例子。

## 协作

改写草稿、补全代码、指出可能缺口。

适合让 AI 做“副驾驶”：拓宽思路、降低上手门槛、加快草稿迭代。

# AI 幻觉：数学里最危险的不足

## 看起来像证明 不等于就是证明

- 编造不存在的定理名或引用。
- 跳过关键引理，把直觉当结论。
- 符号、条件、量词悄悄改变。
- 给出能读顺但不能检查的推理。

## 为什么会发生？

- 语言模型优先生成“像真的文本”。
- 它不天然维护全局数学依赖。
- 没有外部检查时，错误可能很流畅。

因此： AI 输出要当作候选草稿，而不是最终证据。

# 应对幻觉：把 AI 输出变成可检查对象

1

## 问清目标

先写出精确  
statement，确认变  
量、条件、量词。

2

## 拆小步

要求 AI 给出  
lemma 列表，而不  
是一整段长证明。

3

## 查来源

定理名、定义、API  
用 #check / #find /  
文档核对。

4

## 机器检查

把关键步骤放进  
Lean，让 kernel 给  
最后判断。

课堂工作流：AI 负责提出候选，学生负责理解与拆解，Lean 负责机械检查。

# 现在开始写 Lean

先不要急着证明复杂定理。

我们从一个问题开始：Lean 如何看待“表达式”？

```
#check Nat  
#check 37  
#eval 2 + 3
```

核心习惯：每写一行，看 `Infoview` 的目标和类型。

# 进入 Lean 前：类型论的三个核心想法

Lean 的证明不是自然语言段落，而是在类型论里构造对象。先记住这三个直觉就够了。

## 1 表达式都有类型

```
#check 2  
-- 2 : Nat
```

```
#check Nat  
-- Nat : Type
```

## 2 函数是类型之间的箭头

$f : A \rightarrow B$

给一个  $A$ ，返回一个  $B$ 。  
Lean 会检查输入和输出是否匹配。

## 3 命题也是类型

$P : \text{Prop}$   
 $h : P$

读作： $h$  是命题  $P$  的一个证明。  
这是 Curry-Howard 的入口。

接下来写 Lean 时，先问：这个表达式的类型是什么？当前目标要构造什么类型的对象？

# 类型判断： $a : A$ 是 Lean 的基本句式

## term : type

读作：表达式  $a$  具有类型  $A$ 。

Lean 首先检查类型；如果无法赋予类型，表达式会在真假或证明之前被拒绝。

**类型错误不是 False**

它表示表达式没有通过 elaborator  
或 type checker。

```
#check 42
#check Nat
#check Type
```

```
#check 42 + 1
-- #check 42 + "hi"
-- type error, not a false theorem
```

课堂读法：看到冒号，先读“具有类型”；不要先把它想成集合属于关系。

# 第一原则：每个表达式都有类型

```
#check Nat  
#check 37  
#check "hello"  
#check true
```

## Lean 回答

- `Nat : Type`
- `37 :  $\mathbb{N}$`
- `"hello" : String`
- `Bool.true : Bool`

**#check 问类型； #eval 问计算结果。**

# #check / #reduce / #print : 三种与 Lean 对话

---

**#check**

问类型

这个名字 / 表达式具有什么  
类型？

**#reduce**

问计算

把表达式规约到可见的  
normal form 。

**#print**

问环境

这个定义在环境里究竟存了  
什么？

```
def onePlusOne : Nat := 1 + 1
#check onePlusOne
#reduce onePlusOne
#print onePlusOne
```



## 类型也有类型： universe 的小图景

```
#check Nat      -- Nat : Type
#check Prop     -- Prop : Type
#check Type     -- Type : Type 1
#check Type 1   -- Type 1 : Type 2
```

**Sort 0 = Prop**

**Sort 1 = Type**

**Sort 2 = Type 1**

### 为什么要分层？

如果存在“所有类型的类型”  $U : U$ ，就会把自指悖论带回来。Lean 用  $\text{Type } u : \text{Type } (u+1)$  分层，避免 Russell 式问题。

# Russell 式问题：为什么不能有“ $U : U$ ”

## 朴素想法

如果允许一个“所有集合的集合”或“所有类型的类型”把自己也装进去，自指会变成危险的表达式。

$$R = \{x \mid x \notin x\}$$

$R$ ：所有“不属于自己”的对象

问： $R$  属于自己吗？

假设  $R \in R$

定义要求： $R \notin R$

得到矛盾

假设  $R \notin R$

定义要求： $R \in R$

还是矛盾

## Lean 的处理

```
#check Type
-- Type : Type 1
```

```
#check Type 1
-- Type 1 : Type 2
```

**Type  $u : \text{Type } (u + 1)$**

不存在单一宇宙满足  $U : U$ 。

要点：类型分层不是技术细枝末节，而是为了避免自指悖论导致系统不一致。

## 函数：Lean 的基本积木

```
def double (n : Nat) : Nat :=  
  n + n
```

```
#check double  
#eval double 21
```

def

给对象命名

(n : Nat)

输入参数

: Nat

返回类型

n + n

函数体

# 匿名函数与高阶函数

```
#check fun x : Nat => x + 1
```

```
def applyTwice (f : Nat -> Nat) (x : Nat) : Nat :=  
  f (f x)
```

```
#eval applyTwice (fun n => n + 1) 10
```

## 高阶函数

函数也可以作为输入。  
这就是函数式编程的核心味道。

## 多参数函数其实是“一次吃一个参数”

```
def add3 (a b c : Nat) : Nat :=  
  a + b + c
```

```
#check add3  
#check add3 1  
#check add3 1 2  
#check add3 1 2 3
```



**add3 1 仍然是函数：它还在等 2 个参数。**

## 隐式参数：Lean 会帮你推断

```
def myConst {A B : Type} (a : A) (_ : B) : A :=  
  a  
  
#eval myConst 5 "ignored"
```

**{A B : Type}**

花括号参数通常由  
Lean 从上下文推  
断。

初学建议：看不懂时先把隐式参数当成“Lean 自动补的类型信息”。

## Lean 的三种参数括号决定谁来填写参数

看函数、定理或结构字段时，括号不是装饰，而是在告诉 Lean：这个参数由用户写、由系统推断，还是由 `typeclass` 搜索。

**`(x :  $\alpha$ )`**

显式参数。调用时通常要写出来：它是数学对象本身，例如  $n$ 、 $x$ 、一个集合  $s$ 。

**`{ $\alpha$  : Type}`**

隐式参数。Lean 通常能从后面的参数或目标里推出它；类型参数最常这样写。

**`[Monoid  $\alpha$ ]`**

实例参数。Lean 到 `instance` 数据库里找结构，例如群、环、拓扑空间、`DecidableEq`。

```
#check List.length  
#check @List.length
```

```
#check add_zero  
#check @add_zero
```

- 普通 `#check List.length` 会隐藏能推断的参数。
- `@List.length` 会把隐式参数也显示出来。
- `@add_zero` 还能看到 `[AddZeroClass M]` 这样的实例参数。

# 应用函数时，多数隐式参数交给 Lean 推断

初学时先按普通数学写法调用；推断失败时，再用 @、命名参数或 \_ 把缺失信息补给 Lean。

```
def same {α : Type} (x : α) : α := x

#check same 3
#check @same Nat 3

example {α : Type} [AddMonoid α] (x : α) :
  x + 0 = x := by
  simp

example : same (α := Nat) 3 = 3 := by
  rfl
```

## 普通调用

same 3 中，Lean 从  $3 : \text{Nat}$  推断出  $\{\alpha := \text{Nat}\}$ 。

## 强制展开隐式参数

@same Nat 3 让隐藏的  $\{\alpha\}$  显式出现，适合调试和教学。

## 实例参数的场景

[AddMonoid α] 告诉 Lean 去找加法单位元结构；代数、序、拓扑中会反复出现。

常用补法：@foo 显示所有隐式参数；foo (α := Nat) 指定名字；foo \_ x 让 Lean 再试着填一个洞。



# $\Pi$ 、 $\forall$ 与 fun：依赖函数的同一条直觉

```
#check ((n : Nat) -> Fin (n + 1))  
#check ( $\forall$  n : Nat, n = n)
```

```
example : ( $\forall$  n : Nat, n = n) := by  
  intro n  
  rfl
```

$\Pi x : A, B x$       输出类型依赖输入  $x$

$\forall x, P x$       Prop 输出时显示成全称量词

**fun**  $x \Rightarrow \dots /$       构造依赖函数或全称证明  
**intro**  $x$

把  $\forall n, P n$  当成“输入任意  $n$ ，输出  $P n$  的证明”的函数类型来读，`intro` 的意义会自然很多。

# 从 Type 到 Prop

数据有类型。  
命题也有类型。  
证明就是命题类型里的一个项。

$2 + 2 = 4$

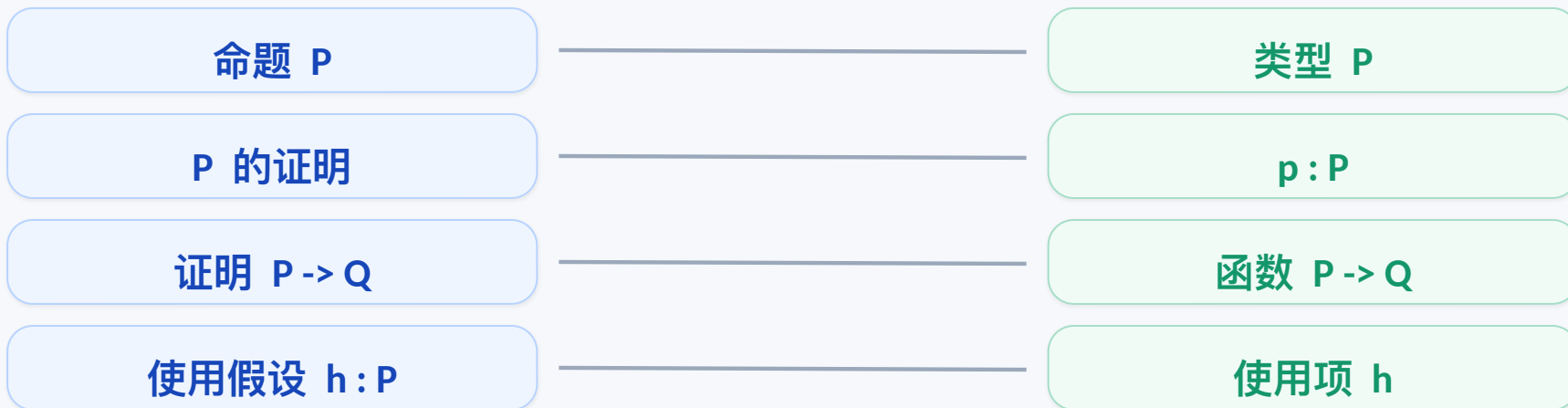
:

**Prop**

## Curry-Howard 对应：具体表述

命题  $P$  可以看成是一个类型  
 $P$  的证明，就是类型  $P$  中的一个项

所以 Lean 证明目标  $P$ ，本质上是在构造一个类型为  $P$  的表达式。



读 Lean 时，把冒号读成“具有类型”： $h : P$  也可以读作“ $h$  是  $P$  的一个证明”。

# Curry-Howard 字典：逻辑联结词怎么变成构造？

| 逻辑                | 证明构造（类比）                | 证明时要做什么          |
|-------------------|-------------------------|------------------|
| True              | True.intro              | 只有一个平凡证明         |
| False             | 无构造子                    | 没有任何项            |
| $P \rightarrow Q$ | fun hP => hQ            | 给 P 的证明，返回 Q 的证明 |
| $P \wedge Q$      | And.intro hP hQ         | 同时给出 P 与 Q 的证明   |
| $P \vee Q$        | Or.inl hP / Or.inr hQ   | 给出左边或右边的证明       |
| $\forall x, P x$  | fun x => hx             | 对任意 x 构造 P x 的证明 |
| $\exists x, P x$  | $\langle x, hx \rangle$ | 给 witness 和性质证明  |

这是类比，不是把 Prop 的命题等同于 Type 的数据；tactic 的任务是构造能被内核检查的证明项。

## 在 Lean 里看见 Curry-Howard

```
example (P Q : Prop) : P -> Q -> And P Q := by
  intro hP
  intro hQ
  exact And.intro hP hQ
```

```
#check And.intro
-- And.intro : P -> Q -> P ∧ Q
```

### **intro hP**

把 P 的证明放入上下文

### **And.intro hP hQ**

把两个证明组装成 And P Q 的证明

因此 tactic 不是魔法命令：它们是在帮助我们构造某个目标类型的项。

## 第一个证明：把目标交给 `exact`

```
example (P : Prop) (h : P) : P := by  
  exact h
```

### Infoview 直觉

上下文:  $h : P$   
目标:  $P$

`exact h` 关闭目标

`exact` 的意思：我已经有一个类型正好等于目标的证明。

## intro : 证明 implication 和 forall

---

```
example (P : Prop) : P -> P := by
  intro h
  exact h
```

目标  
 $P \rightarrow P$

上下文  $h : P$   
目标  $P$

intro 把“假设”放进上下文，把目标往后推进。

## apply : 倒着使用定理

```
example (P Q : Prop) (hPQ : P -> Q) (hP : P) : Q := by
  apply hPQ
  exact hP
```

目标 Q

apply hPQ

新目标 P

apply 的感觉：我可以把目标 Q 还原成它的前提 P。



## constructor : 把 and 目标拆成两半

```
example (P Q : Prop) (hP : P) (hQ : Q) : And P Q := by
  constructor
  exact hP
  exact hQ
```



## cases : 拆开假设或分类讨论

```
example (P Q : Prop) : Or P Q -> Or Q P := by
  intro h
  cases h with
  | inl hP => exact Or.inr hP
  | inr hQ => exact Or.inl hQ
```

两种情况

左边成立  
或  
右边成立

**cases 的感觉：把一个复杂假设拆成可处理的分支。**

## 否定、False 与 contradiction

```
example (P : Prop) (hP : P) (hNotP : ¬ P) : False := by
  contradiction

example (P : Prop) (hP : P) (hNotP : P → False) : False := by
  contradiction
```

 $\neg P$ 

就是  
 $P \rightarrow \text{False}$

False 是没有构造子的命题；同时有  $P$  和  $\neg P$  时，contradiction 会直接找出矛盾。

## exists : 先给 witness , 再证明性质

```
example (m : Nat) : Exists (fun n : Nat => 0 + n = m) := by  
  use m  
  simp
```

**use m**

提供存在性证明  
中的具体对象

存在命题的证明 = 一个对象 + 它满足性质的证明。

## 定义相等：rfl 背后的归约

两个表达式定义相等，是指 Lean 内核把它们展开定义、执行计算后，视为同一个对象。

```
def onePlusOne : Nat := 1 + 1

#reduce (fun n : Nat => n + 1) 3
-- 4
#reduce 2 + 3
-- 5
#reduce onePlusOne
-- 2
```

函数应用会计算 (fun n => n + 1) 3

定义可以展开 onePlusOne -> 2

算术会规约 2 + 3 规约成 5

两边规约后相同 rfl 关闭目标

## rfl : 定义展开后的同一件事

```
example (a : Nat) : a + 0 = a := by  
  rfl
```

```
example (a : Nat) : 0 + a = a := by  
  exact Nat.zero_add a
```

rfl

$a + 0$  : 定义等价  
 $0 + a$  : 需定理证明

rfl 只关闭“定义展开 / 计算后完全相同”的目标； $0 + a = a$  需要 `Nat.zero_add` 这样的定理。

## rfl 失败时：它告诉你什么？

```
example : 2 + 3 = 5 := by
  rfl

-- example : 2 + 2 = 5 := by
--   rfl

example (a : Nat) : 0 + a = a := by
  exact Nat.zero_add a
```

### 失败含义

计算 / 展开后两边仍不是同一个表达式。

### 下一步怎么做

#reduce 看两边  
找定理 Nat.zero\_add  
用 rw / simp / unfold

把 rfl 读成“请内核按定义比较”，而不是“帮我搜索一个证明”。

## rw : 按指定等式改写

```
example (a b c : Nat) (h : a = b) : a + c = b + c := by  
  rw [h]
```

```
example (a b c : Nat) (h : a = b) : c + b = c + a := by  
  rw [h.symm]
```

**rw [h]**

把 a 改成 b

**h.symm**



## simp : 自动做标准化简

```
example (a : Nat) : a + 0 = a := by  
  simp
```

```
def square (n : Nat) : Nat :=  
  n * n
```

```
example : square 3 = 9 := by  
  simp [square]
```

**simp**

使用化简库  
也可以额外展开  
定义

## rw 和 simp 怎么选?

rw [h]

你知道要用哪个等式

精确、可控

simp

目标包含标准化简

自动、快速

simp [h, f]

需要额外假设或展开定义

把信息喂给 simp

# 课堂现场路线

```
import Mathlib

#check Nat
#eval 2 + 3

def double (n : Nat) : Nat := n + n
#eval double 21

example (P : Prop) : P -> P := by
  intro h
  exact h
```

- 先看类型
- 再写函数
- 最后进入证明模式

## 学生最该记住的 6 个动作

---

**#check**

看类型

**#eval**

算结果

**intro**

引入假设 / 变量

**exact**

交出证明

**constructor / cases**

拆目标 / 拆假设

**rw / simp**

改写 / 化简

## 课后练习：把目标变化写下来

```
example (P Q R : Prop) :  
  (P -> Q) -> (Q -> R) -> P -> R := by  
  sorry  
  
example (a b : Nat) (h : a = b) : a + a = b + b := by  
  sorry
```

### 作业方法

每写一行 tactic，记录 Infoview 的目标如何变化。

Lean 入门不是背命令，而是学会读目标、选动作、确认目标变化。